



University
of Glasgow

W5-02: Distributed Storage

COMPSCI4064 (H) and COMPSCI5088 (M)

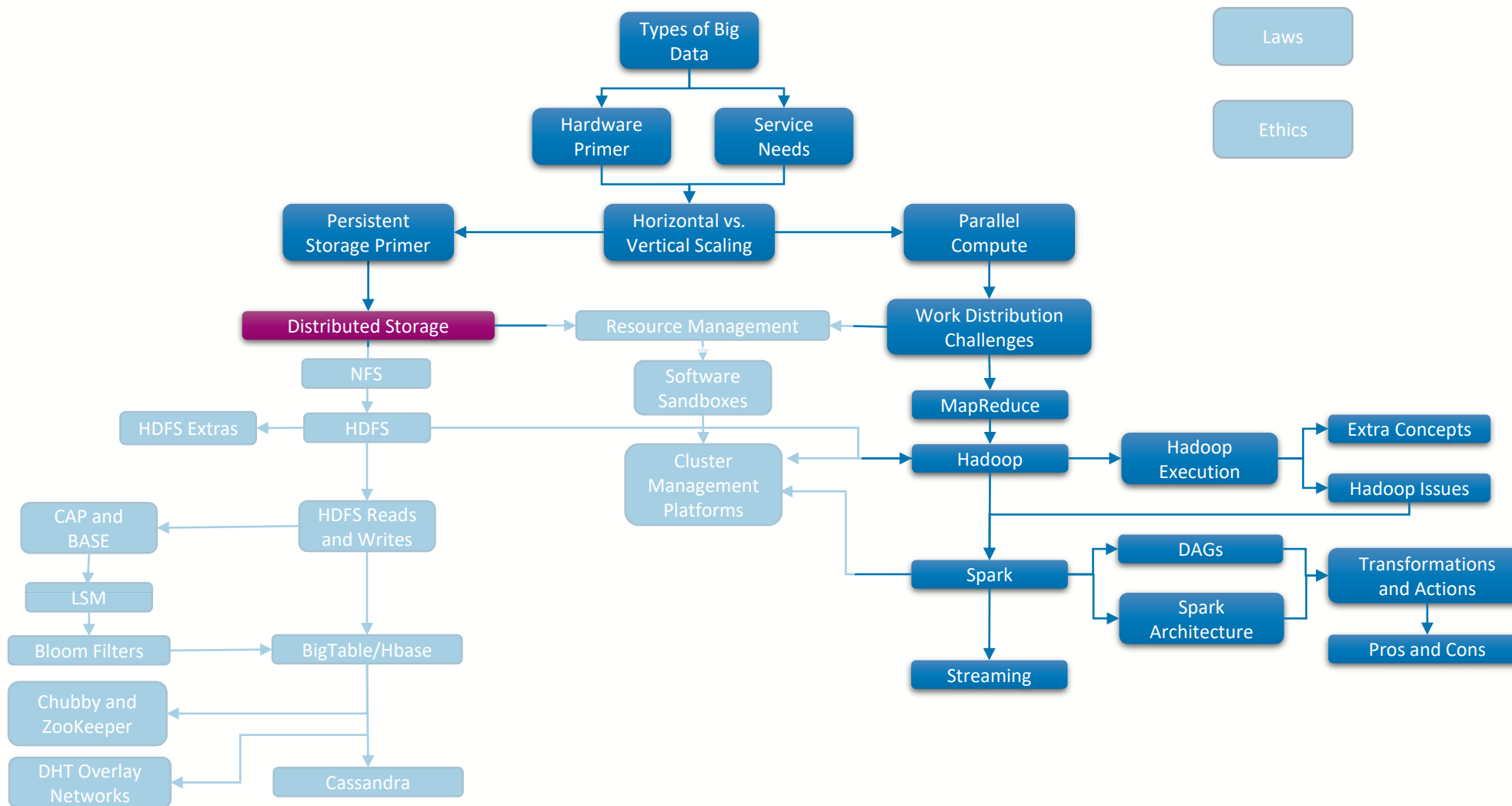
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



Where Are We?





File Systems

- When we talk about distributed storage, what we are really discussing is distributed File Systems
 - A file system *stores and organizes data*
- What do you expect of a file system?
 - A *namespace* (filenames, directory names, etc.)
 - A *storage format* (how files are organised)
 - A *mapping from filenames to storage* (an index)
 - *Read/write access paths* and *API implementation* (a way to actually read and write to the file system)



What is Distributed Storage?

- Distributed storage exists for the same reason that distributed compute exists
 - We need **more hardware capacity** to serve the **data needs** of our programs
 - There are limits to what we can fit in a single machine, and also we can saturate the internal communication links within a machine (see host interfaces)
- So we should **distribute our available storage media across physical machines**
 - The higher the degree of compute parallelism we have the more data storage capacity we need
- For most Big Data applications this is performed via a **distributed file system (DFS)**
 - (The alternative would be to use a distributed database)



Motivations for Distributed Storage

- **Resiliency**

- Data is often mission critical, so we want systems where data is replicated such that a single failure does not result in data loss and a wider system failure

- **Speed**

- The more physical storage devices, the more parallel reads/write we can do, and hence maximum read/write capacity increases

- **Scalability**

- Like with compute, we can theoretically infinitely scale storage in a horizontal manner

- **Data Locality**

- If data is replicated to multiple locations, then we will often be able to schedule work close to that data



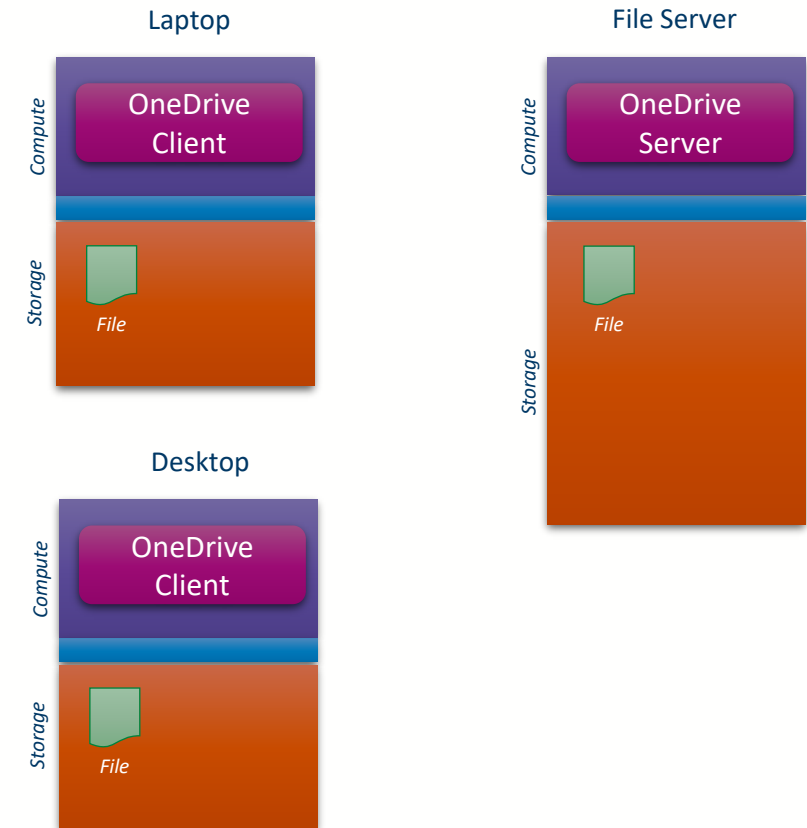
What is the Main Issue with Distributed Storage?

- There is **no 'best solution' for distributed storage**, different architectures/solutions have different advantages and disadvantages
- We end up making a **trade-off** between:
 - **Reliability**: Users expect storage systems to always return the correct and most up-to-date data. If we have multiple copies of a file held in different places, and we want to update that file, how do we ensure that all copies are up-to-date and how long can we wait for this?
 - **Availability**: People count on file systems to always be available, if a data read fails, most applications just stall. But drives sometimes return bad data or otherwise fail.
 - **Performance**: As we distribute data more, we need to rely on the network to transfer data between machines, which involves both the raw data and metadata about what is being held. Performance can vary greatly depending on what type of data is being stored and retrieved (small vs. large files)
 - **Capacity**: As we add more replicas and error checking we expend more space, adding cost



A Simple Example: OneDrive

- Lets imagine a very simple distributed file system (DFS), where files are simply mirrored
- If I write a file locally, it gets copied over the network to the remote mirror

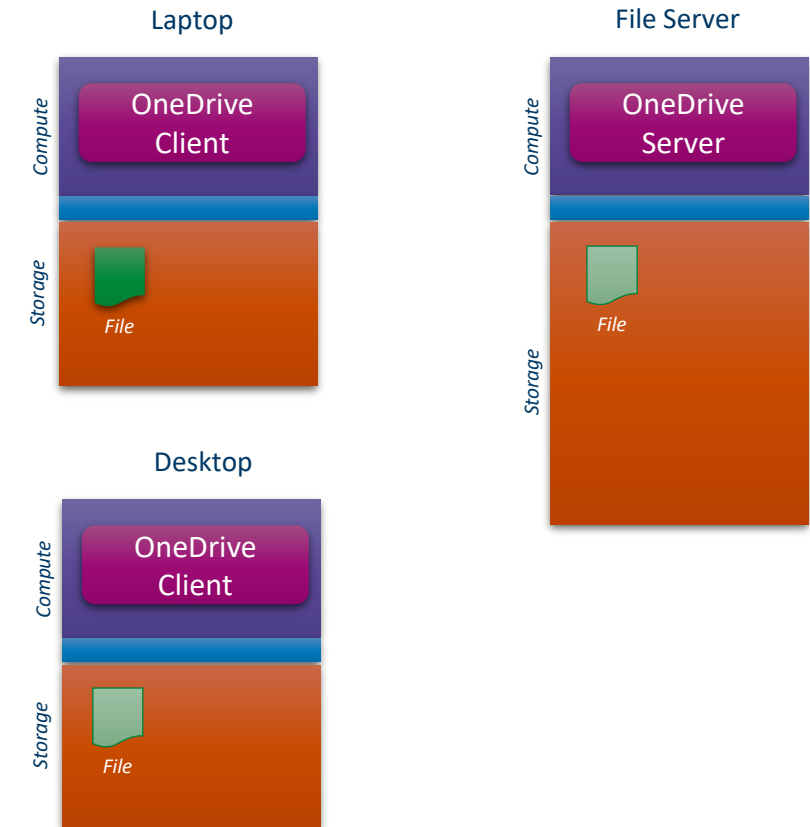




A Simple Example: OneDrive

- Lets imagine a very simple distributed file system (DFS), where files are simply mirrored
- If I write a file locally, it gets copied over the network to the remote mirror

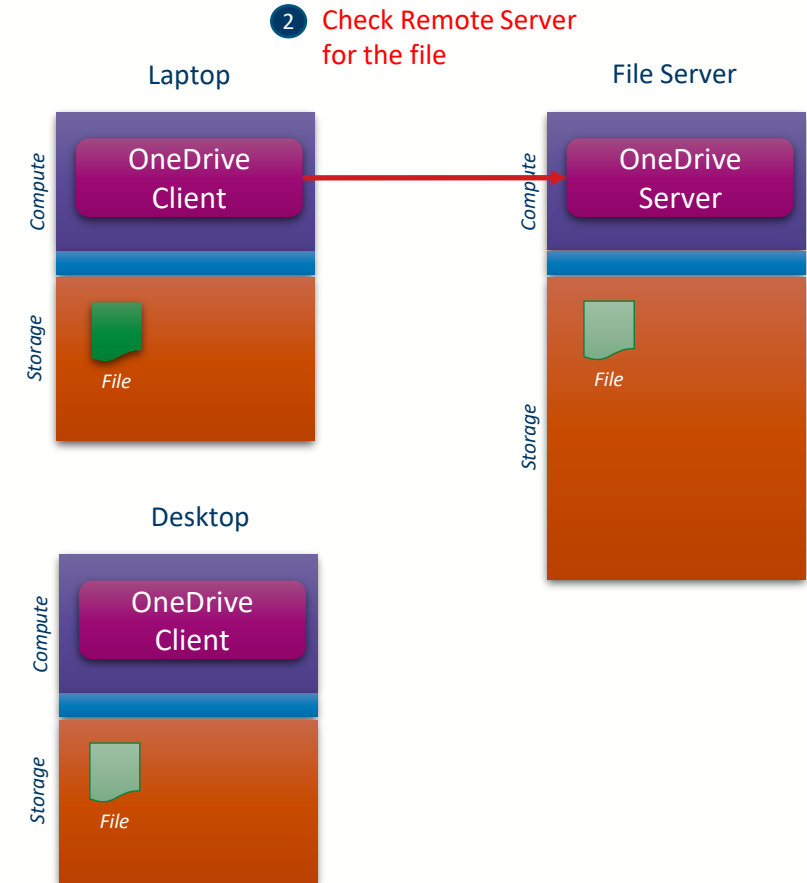
1 Change Gets Detected





A Simple Example: OneDrive

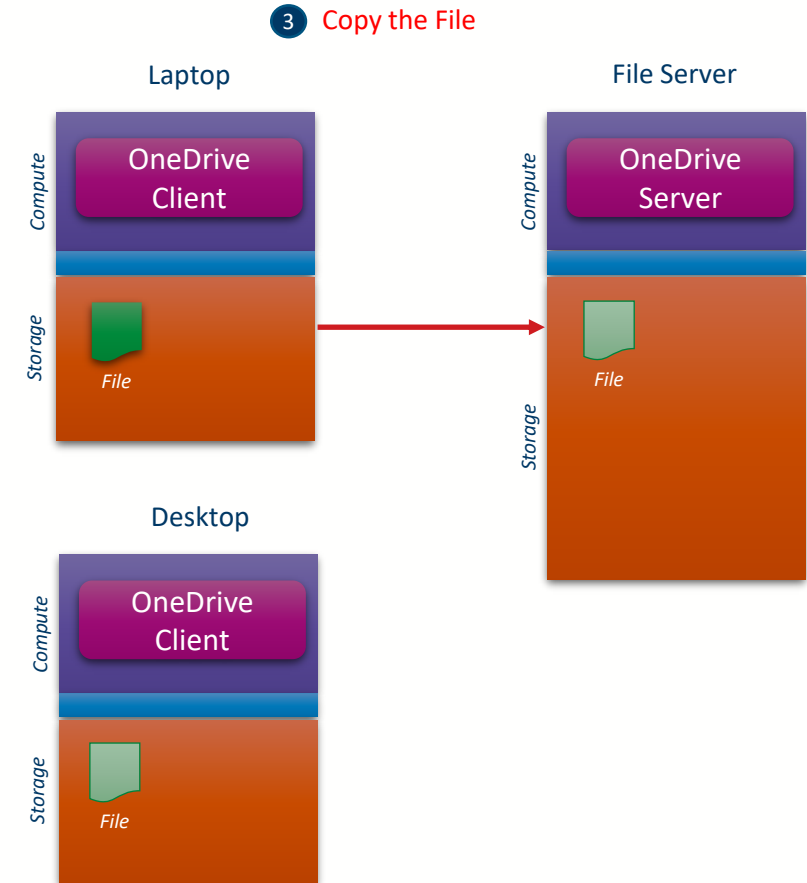
- Lets imagine a very simple distributed file system (DFS), where files are simply mirrored
- If I write a file locally, it gets copied over the network to the remote mirror





A Simple Example: OneDrive

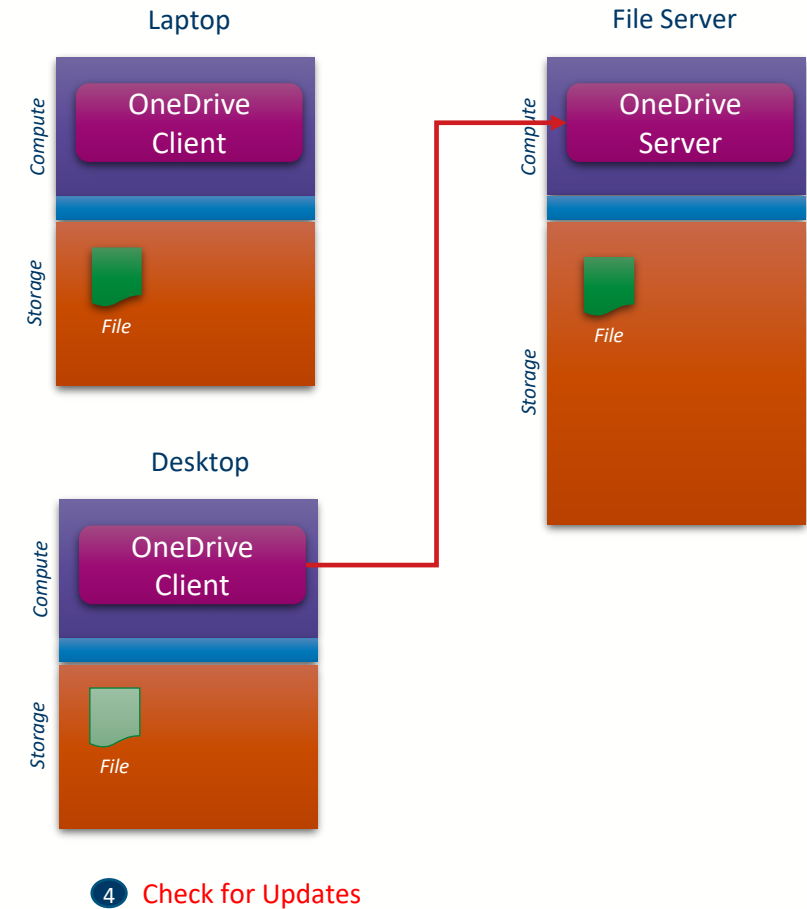
- Lets imagine a very simple distributed file system (DFS), where files are simply mirrored
- If I write a file locally, it gets copied over the network to the remote mirror





A Simple Example: OneDrive

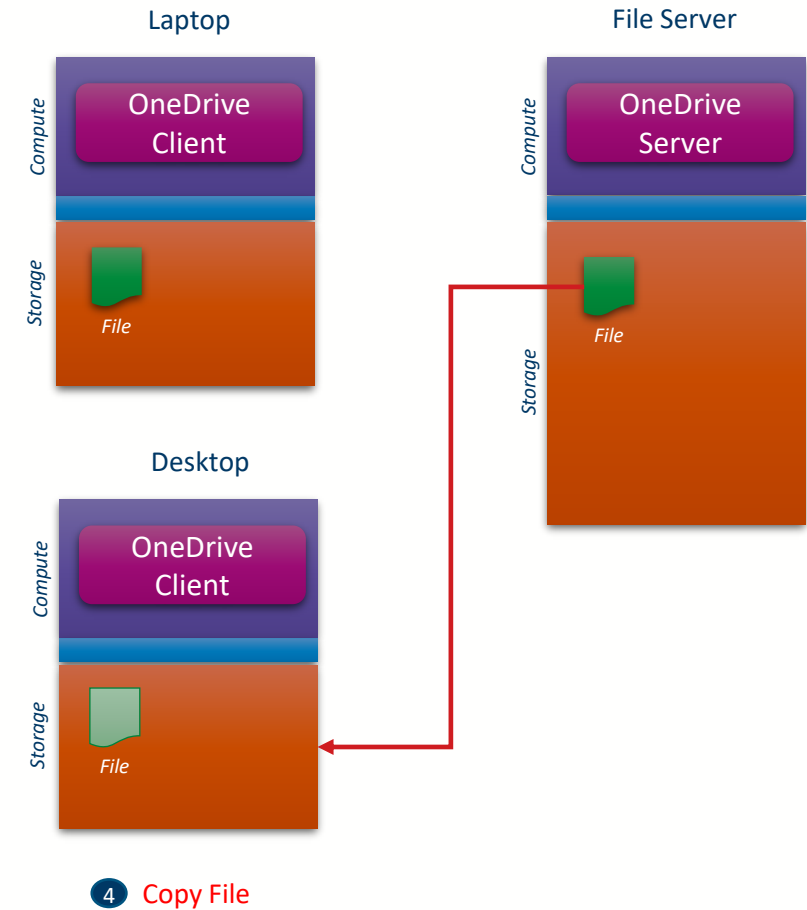
- Lets imagine a very simple distributed file system (DFS), where files are simply mirrored
- If I write a file locally, it gets copied over the network to the remote mirror





A Simple Example: OneDrive

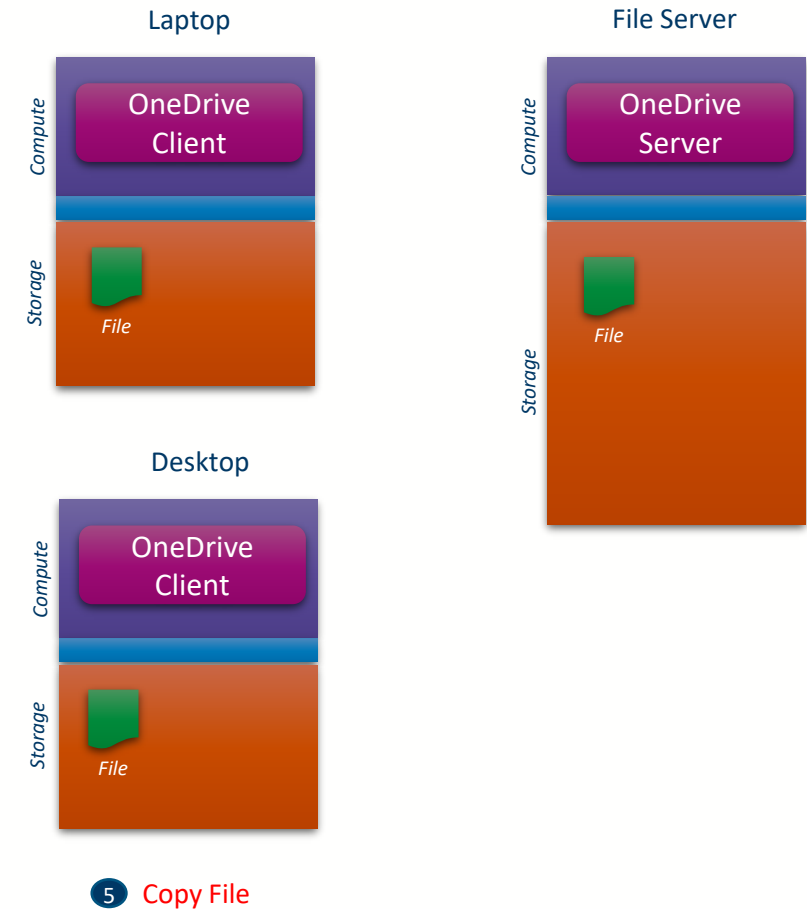
- Lets imagine a very simple distributed file system (DFS), where files are simply mirrored
- If I write a file locally, it gets copied over the network to the remote mirror





A Simple Example: OneDrive

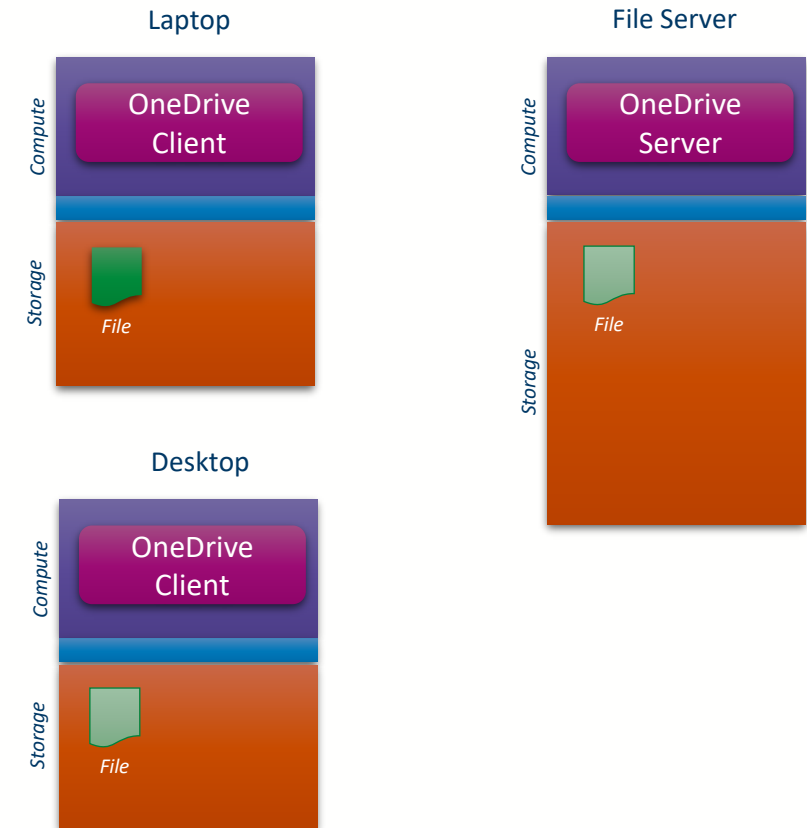
- Lets imagine a very simple distributed file system (DFS), where files are simply mirrored
- If I write a file locally, it gets copied over the network to the remote mirror





A Simple Example: OneDrive

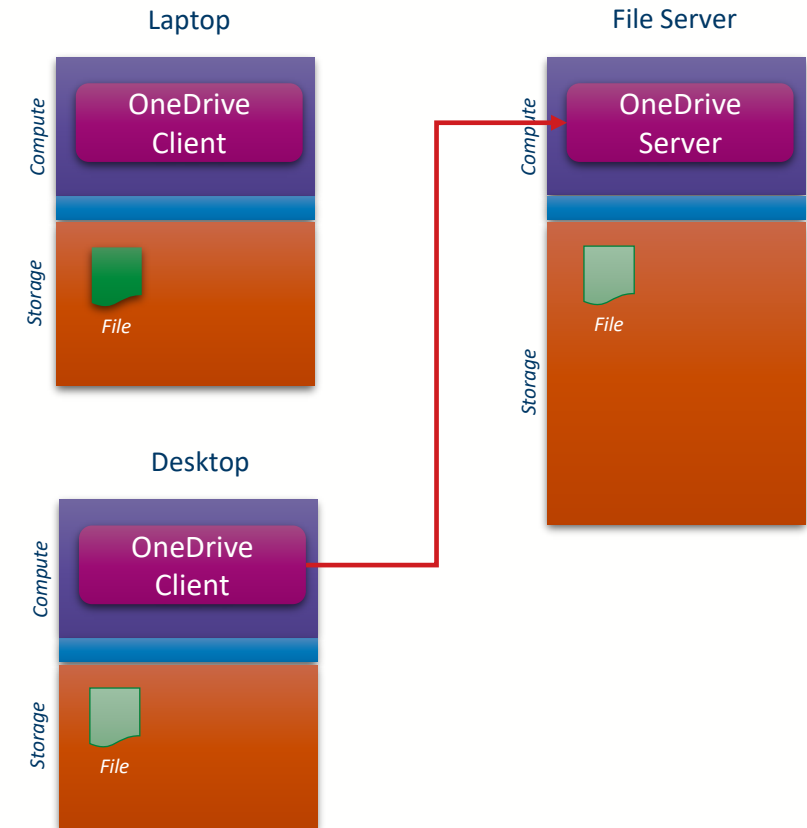
- Now let's imagine all the ways this can go wrong...





A Simple Example: OneDrive

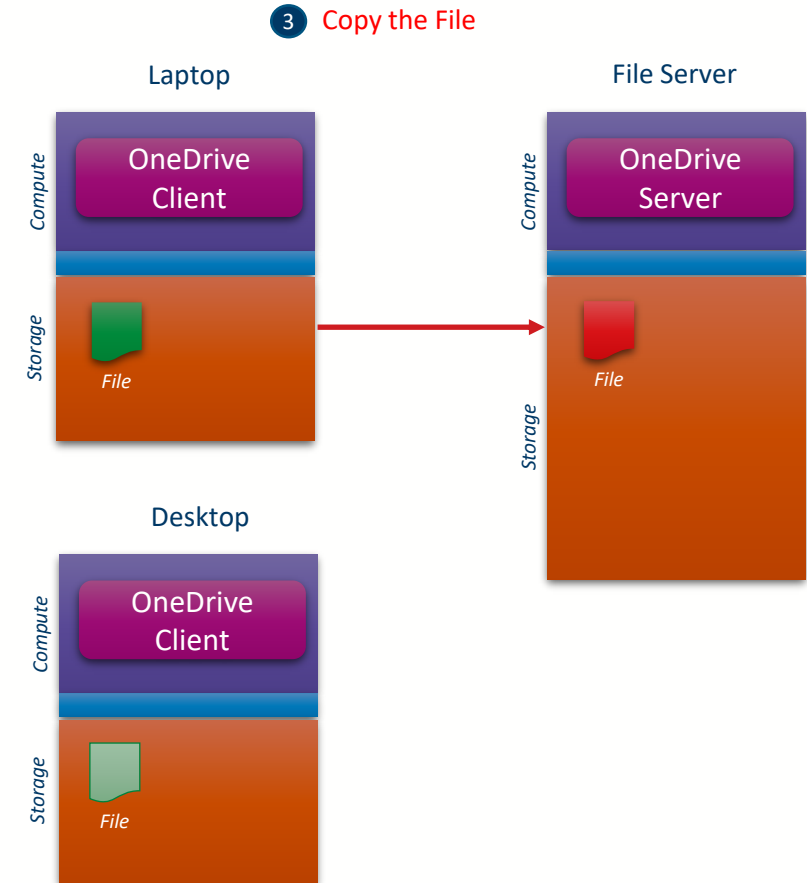
- Now let's imagine all the issues and ways this can go wrong...
1. It can take a while before the **client identifies** that a **file has changed**
 - If another client reads the remote file it will be out-of-date





A Simple Example: OneDrive

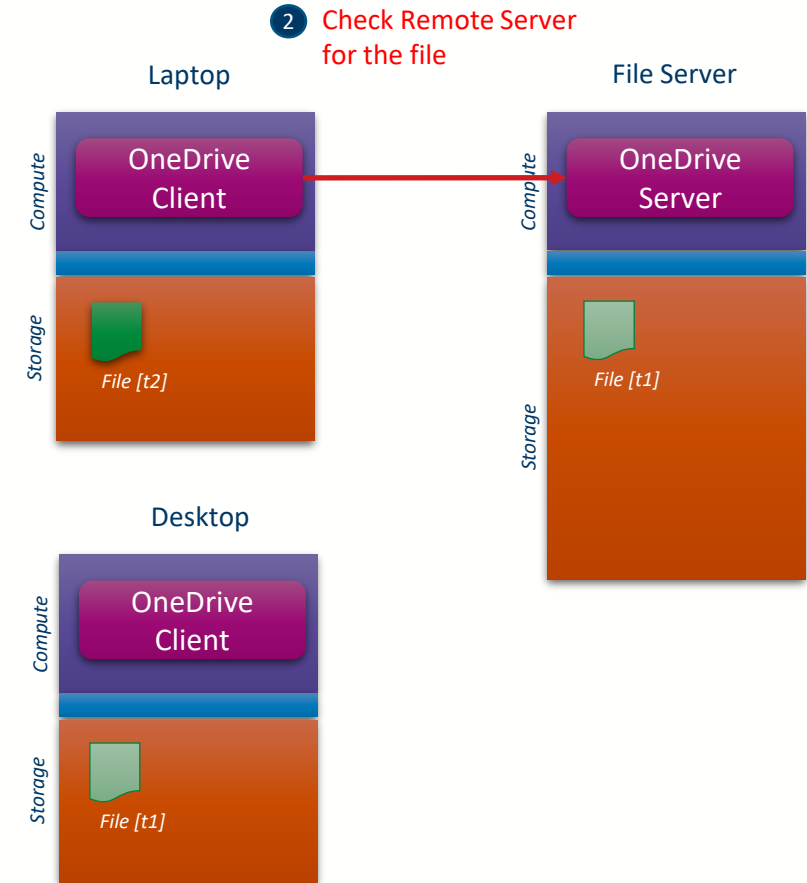
- Now let's imagine all the issues and ways this can go wrong...
1. It can take a while before the **client identifies** that a **file has changed**
 - If another client reads the remote file it will be out-of-date
 2. If there is a **disk read error** then a **corrupted file will be uploaded**, replacing the last good version on the remote





A Simple Example: OneDrive

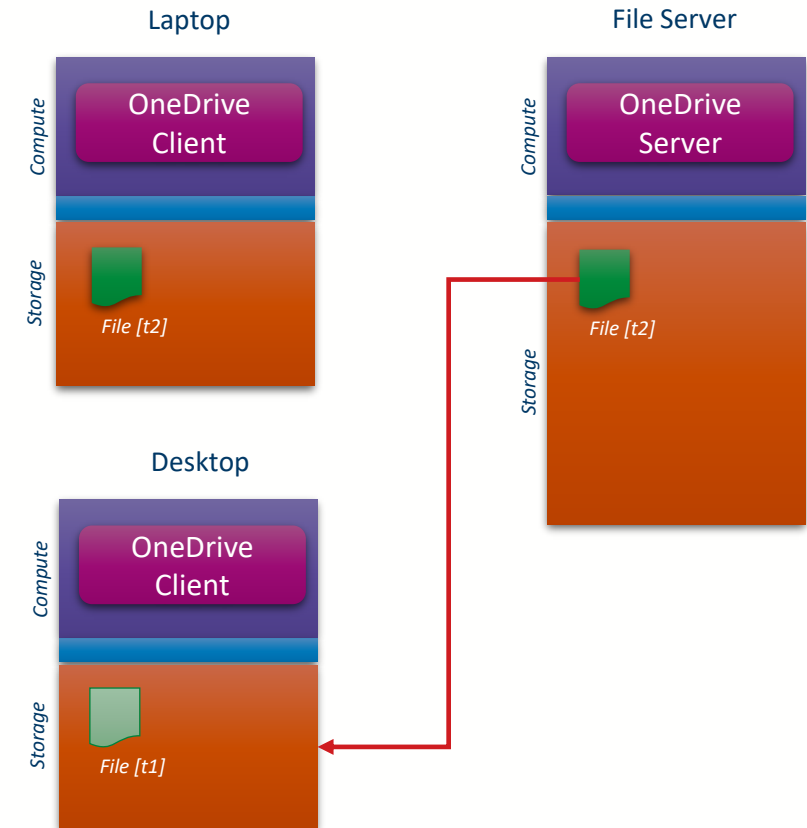
- Now lets imagine all the issues and ways this can go wrong...
1. It can take a while before the **client identifies** that a **file has changed**
 - If another client reads the remote file it will be out-of-date
 2. If there is a **disk read error** then a **corrupted file will be uploaded**, relacing the last good version on the remote
 3. If the server already has a copy of the file **how does it know whether the new local file is newer?**
 - Usually via a last write timestamp
 - What happens if the laptop or server clock time is wrong?





A Simple Example: OneDrive

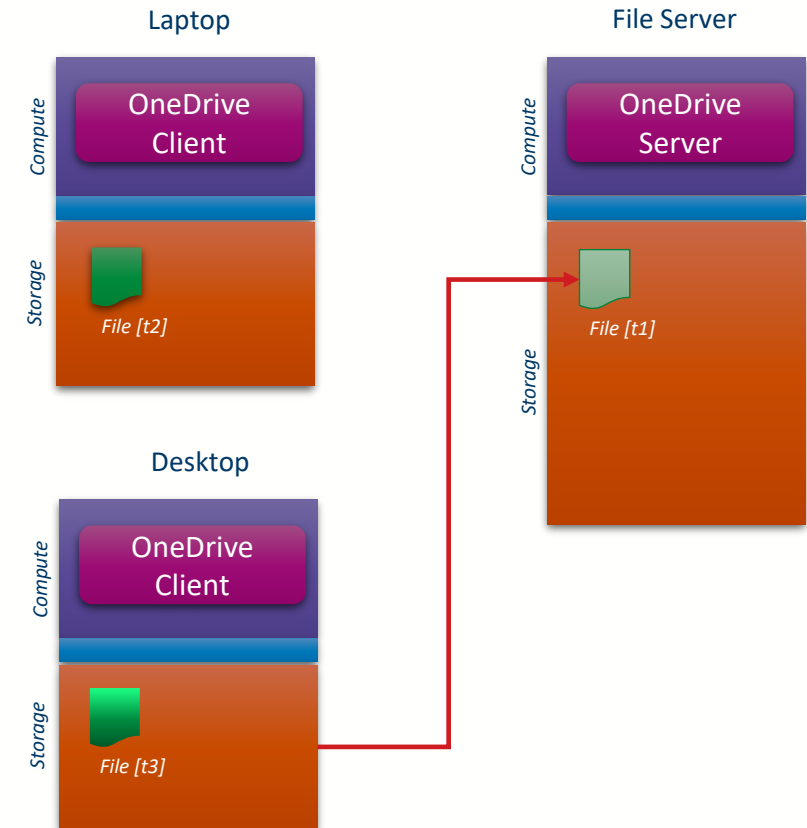
- Now lets imagine all the issues and ways this can go wrong...
4. Should the server **overwrite** a **client file** if it thinks its version is newer?
- What if the user on the client has the file open?





A Simple Example: OneDrive

- Now lets imagine all the issues and ways this can go wrong...
4. Should the server **overwrite** a **client file** if it thinks its version is newer?
 - What if the user on the client has the file open?
 5. What if files are detected **out of temporal order**?
 - How do we resolve this?
- ... and many more





What this means...

- If we are going to use distributed file systems we need to **understand how they handle data and how they react in a range of circumstances**
 - Otherwise we expose ourselves to a range of risks
 - Having our programs blocked by storage system reads/writes
 - Workers receiving inconsistent data
 - Data being lost
 - Running out of storage space



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow